ServiceRegistry

System Description

Abstract
This document describes the ServiceRegistry core system of the Eclipse Arrowhead 5.2.2 framework, as implemented in Java Spring in `ah5-core-java-spring/serviceregistry`, including the local extensions of this workspace.

Contents

1 Overview **3**
1.1 Significant Prior Art 4
1.2 How This System Is Meant to Be Used 4
1.3 System functionalities and properties 4
1.4 Important Delimitations 5

2 Services **6**
2.1 Produced services 6
2.2 Consumed services 6

3 Security **7**
3.1 Security model 7

4 References **7**

5 Revision History **9**
5.1 Amendments 9
5.2 Quality Assurance 9

1 Overview

This document describes the ServiceRegistry system. The ServiceRegistry is the first core system of an Arrowhead local cloud. It keeps the authoritative register of the devices, systems and service instances of the cloud and of the interface templates that describe how services are reached. Every other system registers itself and its services there and finds the core services through it; orchestrators use it to discover candidate providers.



Figure 1: SysML-style block view of ServiceRegistry with its produced and consumed services.

The rest of this document is organized as follows. In Section 1.1, we reference major prior art capabilities of the system. In Section 1.2, we describe the intended usage of the system. In Section 1.3, we describe fundamental properties provided by the system. In Section 1.4, we describe delimitations of capabilities of the system. In Section 2, we describe the abstract service functions consumed or produced by the system. In Section 3, we describe the security capabilities of the system.

1.1 Significant Prior Art

The ServiceRegistry continues the Arrowhead 4.x ServiceRegistry (with the 4.4 DeviceRegistry and SystemRegistry). Arrowhead 5 adds devices as first-class entries, interface templates with typed properties, service instance identifiers `<System>|<serviceDefinition>|<version>`, metadata requirements with operators, and a restricted discovery policy that leaves orchestration as the normal way to find providers.

1.2 How This System Is Meant to Be Used

At start-up every system calls `systemDiscovery.register` and `serviceDiscovery.register`. Systems that need a core service (identity, serviceOrchestration, authorization, ...) look it up with `serviceDiscovery.lookup`; these services carry the metadata `unrestrictedDiscovery=true`. All other services are found through an orchestrator. Operators and management systems maintain the register with `serviceRegistryManagement`.

1.3 System functionalities and properties

1.3.1 Functional properties of the system

- Register, look up and revoke devices, systems and service instances.
- Validate interfaces against interface templates and their property validators; maintain templates.
- Expire service instances at their `expiresAt` time.
- Restrict direct service discovery to sysops and the systems in `service.discovery.direct.access` (policy `restricted`).
- Refuse registrations of reserved core and support service definitions by other systems than their owners (local extension, finding J9).
- Register additional ingress interfaces for its own services (`service.registry.extra.interfaces`, local extension) and honour `service.interface.policy` for them.

1.3.2 Configuration of system properties

The system is configured with Spring properties (`application.properties`, overridden by environment variables in the container deployment). The properties that shape its behaviour are:

Property	Default	Meaning
`server.port`	8443	HTTPS port.
`authentication.policy`	declared	`declared`, `certificate` or `outsourced` (test cloud: `outsourced`).
`arrowhead.security.profile`	secure	`development` selects the isolated development exception; never set in a deployment.
`management.policy` / `management.whitelist`	authorization	Access to `serviceRegistryManagement` and `generalManagement`.
`service.discovery.policy`	restricted	`open` or `restricted` direct lookup.
`service.discovery.direct.access`	DynamicService Orchestration, ...	Systems with unrestricted lookup.
`service.discovery.interface.policy`	restricted	`restricted`, `extendable` or `open` handling of unknown templates.
`discovery.verbose`	true	Whether lookups may include system and device details.

Property	Default	Meaning
`allow.self.addressing` / `allow.non.routable.addressing`	true / false	Address validation.
`service.address.alias`	address, ...	Interface property keys that hold addresses.
`service.definition.reserved`	(built in)	Overrides of the reserved service definitions (local).
`service.interface.policy`	(from authentication policy)	Security policy of the own interfaces; default `CERT_AUTH` behind TLS, `NONE` for declared identities or without TLS (local).
`service.registry.extra.interfaces`	[]	Additional interfaces of the own services (local).
`enable.blacklist.filter` / `force.blacklist.filter`	false / true	Blacklist check of requesters.
`normalization.mode`	extended	Name normalization (`simple` or `extended`).
`mqtt.api.enabled`	false	Offer the MQTT interfaces.
`sysmonitor.enabled`	false	Publish native sysMonitor observations (test cloud: true).
`arrowhead.ssl.crl-file`	(unset)	CRL checked for every client certificate (local, finding J4).

Secrets (database, keystore and identity passwords, secret keys) have no usable defaults; in the test clouds they are per-cloud secrets mounted as files.

1.3.3 Data stored by the system

Devices and their addresses, systems and their addresses and device assignment, service definitions, service instances with metadata and interfaces, interface templates with property rules, and the system's log entries (MariaDB/MySQL database `ah_serviceregistry`).

1.3.4 Non functional properties

The ServiceRegistry is on the critical path of every start-up and orchestration; it is a single instance per cloud in this deployment. Lookups are database queries without caching. Requests are limited to `max.page.size` records per page.

1.3.5 Stateful or stateless

Stateful: all records are persistent in the database; the service itself keeps no session state.

1.4 Important Delimitations

The ServiceRegistry does not verify that a registered service is reachable, does not authorize consumption and does not orchestrate. It trusts the identity established by its authentication policy. Its own services are registered directly into its database at start-up, not through the HTTP API.

2 Services

This section describes the produced and consumed services. Each produced service is specified by an SD document and one IDD document per interface.

2.1 Produced services

Service	Version	Interfaces	SD	IDD
`deviceDiscovery`	1.0.0	HTTP(S), MQTT(S)	[1]	[2], [3]
`systemDiscovery`	1.0.0	HTTP(S), MQTT(S)	[4]	[5], [6]
`serviceDiscovery`	1.0.0	HTTP(S), MQTT(S)	[7]	[8], [9]
`serviceRegistryManagement`	1.0.0	HTTP(S), MQTT(S)	[10]	[11], [12]
`generalManagement`	1.0.0	HTTP(S), MQTT(S)	[13]	[14], [15]
`sysMonitor`	1.0.0	AMQP(S)	[16]	[17]

The HTTP interfaces are always offered, the MQTT interfaces with `mqtt.api.enabled=true` and the AMQP `sysMonitor` interface with `sysmonitor.enabled=true`. Upstream 5.2.2 also answers `GET <base>/monitor/echo` on some systems; this endpoint is not registered as a service.

2.2 Consumed services

Provider	Service	Operations	Use	Condition
Authentication	identity	identity-login, identity-verify	Own identity token; verification of requester tokens (Authentication's secret key is configured, so most tokens are checked locally).	`authentication.policy=outsourced`
Consumer Authorization	authorization	verify	Management filter decisions.	`management.policy=authorization`
Blacklist	blacklistDiscovery	check	Blacklist filter.	`enable.blacklist.filter=true`
AMQP broker (SysMonitor)	sysMonitor (publisher)	echo, power, status	Own observations.	`sysmonitor.enabled=true`

Except for the bootstrap services (`http.collector.bootstrap.services`), which are looked up in the ServiceRegistry, the system reaches other systems' services through DynamicServiceOrchestration (`http.collector.mode=ORCHESTRATION`, results cached for `http.collector.orchestration.ttl` = 300 s); every such call needs a ConsumerAuthorization grant.

3 Security

The default is the Arrowhead secure mode with HTTPS and X.509 certificates following the Arrowhead certificate profile (`<System>.<Cloud>.<Organization>.arrowhead.eu`). Certificates not issued under the cloud CA are refused by the TLS handshake. The Arrowhead-test-cloud-java deployment runs the secure mode.

The packaged `application.properties` is a development sample that disables TLS and declares identities. The system refuses to start with it, or with any other effective configuration that sets `server.ssl.enabled=false`, `authentication.policy=declared`, `service.authorization.enabled=false` or, while monitoring is on, `sysmonitor.ssl.enabled=false`. The check runs before the system accepts a connection. Only an explicit `arrowhead.security.profile=development` lets such a system start, for an isolated test, and it then logs a non-conformance warning.

3.1 Security model

- **Protocols:** HTTPS (TLS 1.3 and 1.2) with mandatory client certificates (`server.ssl.client-auth=need`); MQTT over TLS; AMQP over TLS for `sysMonitor`.
- **Data protection:** transport encryption by TLS; the database connection uses TLS with server verification in the test cloud. Secrets are kept out of the container environment (Compose file secrets).
- **System authentication:** by the authentication policy: `certificate` (client certificate CN), `outsourced` (identity token verified with Authentication), `internal` (Authentication only) or `declared` (insecure). In this workspace an identity token is accepted only together with the client certificate of the system it was issued to (token-to-certificate binding), and every client certificate is checked against the cloud CA's CRL when `arrowhead.ssl.crl-file` is set (fail closed).
- **Authorization of produced services:** management services are protected by the management filter (`management.policy`: `SYSOP_ONLY`, `WHITELIST` or `AUTHORIZATION` via ConsumerAuthorization); an optional blacklist filter refuses blacklisted requesters. Sysop identities bypass the management filter.
- Under `restricted` discovery, a requester that is neither a sysop nor in `service.discovery.direct.access` only sees services flagged `unrestrictedDiscovery`; other instances are omitted from the result without an error. A system that resolves SimpleStoreServiceOrchestration results with `lookup`, and a Gatekeeper that provides services to other clouds, must therefore be listed. The service definitions of the core and support systems can only be registered by their owners (403 otherwise).

For the Arrowhead certificate profile see github.com/eclipse-arrowhead/documentation.

4 References

[1] J. Delsing, "deviceDiscovery – Service Description (ServiceRegistry)," Eclipse Arrowhead, SD 5.2.2, 2026, serviceRegistry-v5.2.2-deviceDiscovery-SD.pdf.

[2] ——, "deviceDiscovery HTTP/TLS/JSON – Interface Design Description (ServiceRegistry)," Eclipse Arrowhead, IDD 5.2.2, 2026, serviceRegistry-v5.2.2-deviceDiscovery-HTTP-IDD.pdf.

[3] ——, "deviceDiscovery MQTT/TLS/JSON – Interface Design Description (ServiceRegistry)," Eclipse Arrowhead, IDD 5.2.2, 2026, serviceRegistry-v5.2.2-deviceDiscovery-MQTT-IDD.pdf.

[4] ——, "systemDiscovery – Service Description (ServiceRegistry)," Eclipse Arrowhead, SD 5.2.2, 2026, serviceRegistry-v5.2.2-systemDiscovery-SD.pdf.

[5] ——, "systemDiscovery HTTP/TLS/JSON – Interface Design Description (ServiceRegistry)," Eclipse Arrowhead, IDD 5.2.2, 2026, serviceRegistry-v5.2.2-systemDiscovery-HTTP-IDD.pdf.

[6] ——, "systemDiscovery MQTT/TLS/JSON – Interface Design Description (ServiceRegistry)," Eclipse Arrowhead, IDD 5.2.2, 2026, serviceRegistry-v5.2.2-systemDiscovery-MQTT-IDD.pdf.

[7] ——, “serviceDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, serviceRegistry-v5.2.2-serviceDiscovery-SD.pdf.

[8] ——, “serviceDiscovery HTTP/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.2.2, 2026, serviceRegistry-v5.2.2-serviceDiscovery-HTTP-IDD.pdf.

[9] ——, “serviceDiscovery MQTT/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.2.2, 2026, serviceRegistry-v5.2.2-serviceDiscovery-MQTT-IDD.pdf.

[10] ——, “serviceRegistryManagement – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, serviceRegistry-v5.2.2-serviceRegistryManagement-SD.pdf.

[11] ——, “serviceRegistryManagement HTTP/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.2.2, 2026, serviceRegistry-v5.2.2-serviceRegistryManagement-HTTP-IDD.pdf.

[12] ——, “serviceRegistryManagement MQTT/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.2.2, 2026, serviceRegistry-v5.2.2-serviceRegistryManagement-MQTT-IDD.pdf.

[13] ——, “generalManagement – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, serviceRegistry-v5.2.2-generalManagement-SD.pdf.

[14] ——, “generalManagement HTTP/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.2.2, 2026, serviceRegistry-v5.2.2-generalManagement-HTTP-IDD.pdf.

[15] ——, “generalManagement MQTT/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.2.2, 2026, serviceRegistry-v5.2.2-generalManagement-MQTT-IDD.pdf.

[16] ——, “sysMonitor – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, serviceRegistry-v5.2.2-sysMonitor-SD.pdf.

[17] ——, “sysMonitor AMQP/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.2.2, 2026, serviceRegistry-v5.2.2-sysMonitor-AMQP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-11-07	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	